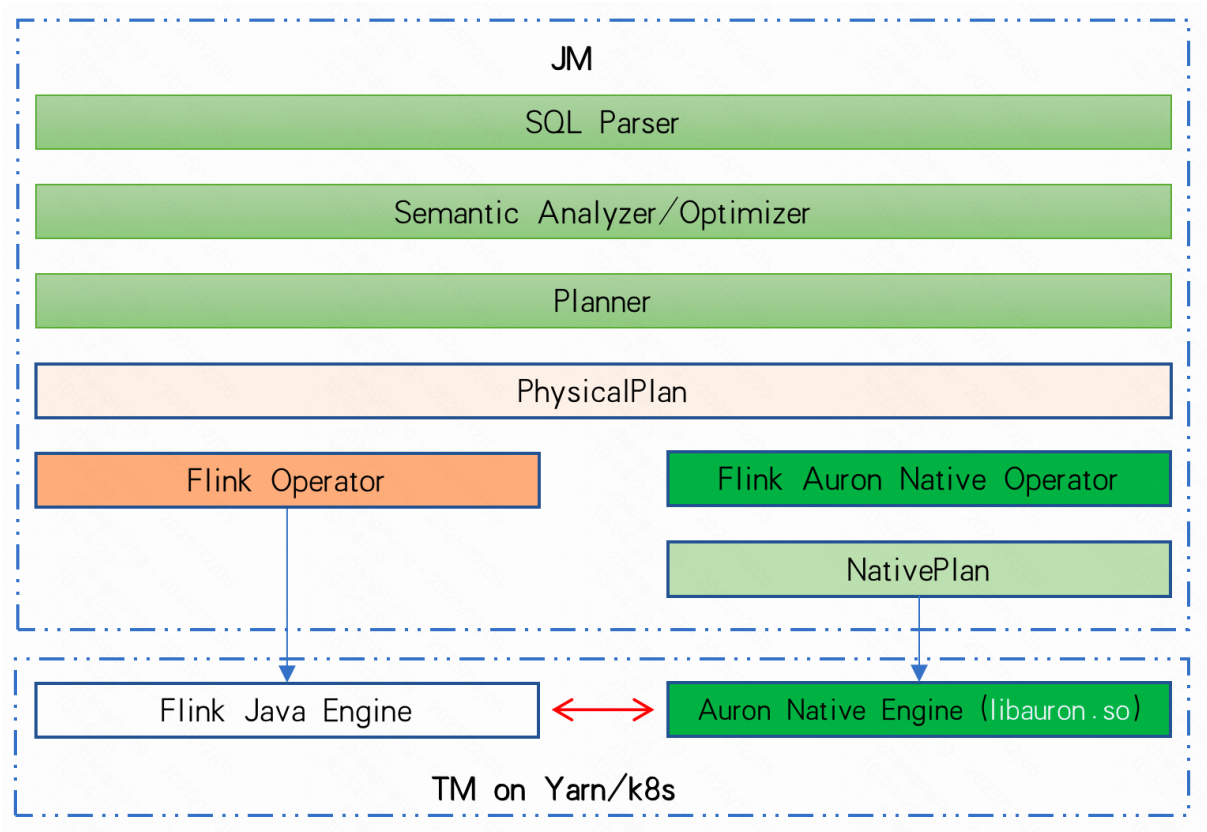
AIP-1: Introduce Flink integration of native engine

Motivation

The main purpose of this AIP is to improve the near-real-time (NRT) experience of Flink. It also helps realize Auron's vision of supporting multiple engines and scenarios.We will explain why we have this decision and the benefits of flink on auron. Usually, we can categorize data processing into **real-time** (in seconds), **near real-time** (in minutes), and **batch** (> hours) processing. Flink is a well-known stream processing system which is good at real-time scenarios. Spark is a well-known batch processing system which is good at offline scenarios. Spark SQL on Auron has demonstrated significant performance advantages in offline scenarios. Flink can provide low latency with the cost of per-record state operation. But users don't need such a low latency in some cases. It would be great if the tolerated delay can be exchanged for a huge increase in throughput. Therefore, we believe that Auron can also achieve the effect of accelerating Flink SQL real-time computing. Of course, implementing Flink SQL on Auron won't happen overnight. We'll need extensive development work and data validation.

Architecture

The diagram below illustrates the architecture for Flink Streaming SQL to collaborate with Auron. The SQL syntax and Planner fully leverage Flink's existing capabilities, ensuring complete transparency to upper-layer users.
Auron translates Flink's physical plan into an equivalent plan recognizable by Auron's native vectorized engine. The FlinkAuronNativeOperator is used to execute the Native Plan.
For operators not yet supported, Flink's original implementation will continue to be used.
Similar to Spark usage, we only need to replace the Flink table-related JARs with the Auron JARs.



Public Interfaces

To identify whether an operator is implemented by Auron Native, we will introduce the following interface:

Introduce SupportsAuronNative, FlinkAuronOperator, FlinkAuronFunction

Since there will be native support implementations at both the Flink Operator and Function levels, the SupportsAuronNative abstraction is introduced.

SupportsAuronNative primarily provides the following information:

- Native Plan
- Return Data Type
- Flink Auron Operator Unique ID

Both FlinkAuronOperator and FlinkAuronFunction implement the SupportsAuronNative interface.

Proposed Changes

Note: This AIP only discusses Flink stream processing implementations and does not cover Flink Batch scenarios.

Flink Node Converter Implementation Details

Only a few nodes are listed here as examples; they will be expanded in actual development.

Introduce FlinkNodeConverterFactory to get node converter instance.

Introduce FlinkNodeConverter / ConverterContext

- Translate Flink Node to Auron's PhysicalPlanNode
- The ConverterContext provides additional information, such as schema details, for future extensibility.

Rewriting the Exec Node in Flink SQL

Flink isn't like Spark has a few open APIs (plugin api, columnar api, rule api). To integrate with Auron, we need to rewrite some core classes in Flink. In the future, we can drive the Flink community to support runtime engine plugins.

We need to rewrite `org.apache.flink.table.planner.plan.nodes.exec.stream.StreamExecCalc` to achieve the goal of making Calc native.

- Generate a complete Auron PhysicalPlanNode
- Generate a FlinkAuronOperator to execute the PhysicalPlanNode

Introduce FlinkAuronConfiguration

Auron acts as a proxy for Flink configuration, enabling it to retrieve any configuration required by Auron through Flink config.

- All configurations used by Auron in Flink scenarios must begin with `flink.`
- Allow users to configure in flink.yaml

Support Functions

Flink offers a wide range of practical built-in functions. For more details, please refer to the link below.
https://nightlies.apache.org/flink/flink-docs-release-1.20/docs/dev/table/functions/systemfunctions/
We can prioritize converting frequently used functions to native implementations.

Support Connectors

Kafka Connector

Implementation of a native Kafka Source and Sink requires the following steps:

- Introduce AuronNativeKafkaDynamicTableFactory implement DynamicTableSourceFactory and DynamicTableSinkFactory
- Introduce AuronNativeKafkaDynamicTableSource implement ScanTableSource
- Introduce AuronNativeKafkaDynamicTableSink implement DynamicTableSink
- Introduce NativeKafkaSourceFunction and implement FlinkAuronFunction interface
- Introduce NativeKafkaSinkFunction and implement FlinkAuronFunction interface
- Rewrite LegacySourceTransformationTranslator to replace StreamSource with FlinkAuronStreamSource
- Rewrite LegacySinkTransformationTranslator to replace SinkOperator with FlinkAuronSinkOperator
- Introduce FlinkAuronStreamSource extends StreamSource and implements FlinkAuronOperator interface
- Introduce FlinkAuronSinkOperator extends SinkOperator and implements FlinkAuronOperator interface

The core logic of NativeKafkaSinkFunction and NativeKafkaSinkFunction is to execute Auron's plan.
The code for native Kafka source and sink is not covered here.

Other Connectors

The implementation approaches for other Flink connectors are similar to Kafka's, so we won't elaborate further. Phase 1 primarily focuses on resolving the implementation of the Kafka connector.

Support Formats

- Json
- Protobuf

Invoke the relevant format's class library to serialize and deserialize data. With out going into excessive detail, the core issue lies in the performance when conve rting row data to Arrow format.

Compatibility, Deprecation, and Migration Plan

There is no compatibility problem in Flink integration.

Implementation Plan

The implementation of Flink streaming integration is planned in three phases.

Phase 1: Stateless Operator Support(Calc Operator)

- Support various operator expressions: such as + - * /
- Support Flink built-in functions: such as unix_timestamp
- Support logical expression
- Support user-defined Flink functions
- Support RowData to Arrow
- Support Native flink kafka connector
- Supports whole stage native operator merging

Phase 2: Stateful Operator Support(Agg Operator)

- Support Flink built-in agg functions
- Support Flink mini-batch one phase agg
- Support Flink mini-batch two phase agg

- Support user-defined Flink agg functions
- Support Flink count distinct
- Support Native State

Phase 3: Stateful Operator Support(Join Operator)

- Support Regular Join
- Support Window Join
- Support other Join

This AIP primarily addresses the implementation of Phase 1.

Test Plan

1. Unit test cases to test methods in auron-flink-extension.

2. Add ITCase to test the correctness of Flink SQL Auron operators.

3. Add E2ECase to test the overall correctness, robustness, and end-to-end semantic compliance of the Flink and Auron integration.

Rejected Alternatives

This direction represents a new area of exploration, so there are no other rejected alternatives.

References

Some experience for flink native engine:

- https://www.alibabacloud.com/blog/flash-a-next-gen-vectorized-stream-processing-engine-compatible-with-apache-flink_602088
- https://zhuanlan.zhihu.com/p/1916129445640648620